

Wie das Automix-Modell funktioniert

Schritt-für-Schritt-Erklärung als Grundlage für das Paper

Projekt *Automatic Mixing*, Deep Learning SS26

Stand: 18. September 2026

Zusammenfassung

Wir stellen ein kleines neuronales Netz vor, das Orchesteraufnahmen automatisch zu einem Stereomix zusammenstellt. Eingabe sind die einzelnen Mikrofonspuren einer Aufnahme – das Hauptsystem und die Stützmikrofone an den Instrumenten. Für jede Spur sagt das Modell zwei Mischpult-Einstellungen vorher: einen Pegel (Gain) und eine Position im Stereopanorama (Pan). Ein eingefrorenes, vortrainiertes Audionetz (VGGish) beschreibt den Klang jeder Spur; ein trainierbares mehrschichtiges Perzeptron mit nur 132 100 Parametern trifft daraus die Entscheidungen. Die Einstellungen werden über ein differenzierbares Pan-Gesetz zu einem Stereosignal gemischt, sodass das Modell allein aus fertigen Mischungen lernt, ohne je die „richtigen“ Reglerstellungen zu sehen. Auf zurückgehaltenen synthetischen Stücken mit bekannter Aufstellung platziert das Modell die Instrumente mit einem mittleren Fehler von $5,8^\circ$ (Rauschgrenze $2,5^\circ$, Mitte-Tipp $17,1^\circ$).

Inhaltsverzeichnis

1 Die Aufgabe: Was heißt hier „automatisch mischen“?	2
2 Überblick: Der Weg durch das Modell	3
3 Schritt 1: Vom Audio zum Klangvektor – der VGGish-Encoder	3
4 Schritt 2: Der Kontext – was spielt sonst noch mit?	4
5 Schritt 3: Die Entscheidung – das MLP	4
6 Schritt 4: Anker – feste Positionen für das Hauptsystem	5
7 Schritt 5: Der Mixer – ein differenzierbares Pan-Gesetz	5
8 Schritt 6: Wie gut ist der Mix? – die Verlustfunktion	6
9 Schritt 7: Das Training	7
9.1 Daten	7
9.2 Ablauf	7
9.3 Nächster Schritt: Die echten Mischungen stärker nutzen	7
10 Schritt 8: Anwendung auf ein neues Stück	8

11 Was das Modell kann – Ergebnisse im Überblick	8
12 Grenzen und offene Punkte	9
13 Vorschlag: Vom Dokument zum Paper	10

1 Die Aufgabe: Was heißt hier „automatisch mischen“?

Bei einer Orchesteraufnahme hängen viele Mikrofone gleichzeitig im Saal:

- **Hauptsystem:** wenige Mikrofone über dem Dirigenten, die das ganze Orchester als Klangbild einfangen, z. B. ein Decca-Tree aus drei Mikrofonen (links, Mitte, rechts) und oft zusätzlich ein Raum-Paar.
- **Stützmikrofone (Spots):** Mikrofone nah an einzelnen Instrumenten oder Gruppen, etwa an den ersten Violinen oder der Harfe.

Der Tonmeister mischt diese Spuren zu einem Stereosignal. Dazu stellt er für jede Spur mindestens ein, *wie laut* sie in den Mix eingeht (Gain), und *wo* sie im Stereobild erscheint (Pan). Genau diese beiden Entscheidungen soll unser Modell übernehmen.

Formal. Gegeben sind N Mono-Spuren $s_1(t), \dots, s_N(t)$ eines Stücks. Gesucht ist für jede Spur ein Gain $g_i \geq 0$ und ein Panoramawinkel $\theta_i \in [0, \pi/2]$ (in Grad: 0° bis 90°), sodass der daraus gemischte Stereomix

$$\hat{y}_L(t) = \sum_{i=1}^N g_i \cos(\theta_i) s_i(t), \quad \hat{y}_R(t) = \sum_{i=1}^N g_i \sin(\theta_i) s_i(t) \quad (1)$$

möglichst gut klingt – im Training heißt das: möglichst nah an einem Referenzmix $y = (y_L, y_R)$ liegt.

Bewusste Vereinfachung. Das Modell wählt *eine* feste Einstellung pro Spur für das ganze Stück. Es gibt keine Pegelautomation, keinen Equalizer, keinen Hall und keine Laufzeitkorrektur. Das macht das Modell klein, die Ausgabe direkt interpretierbar (zwei Zahlen pro Spur, wie an einem Mischpult) und die ganze Kette differenzierbar. Dass die Vereinfachung nicht aus der Luft gegriffen ist, zeigen die echten Mischungen des Spheres-Datensatzes: Sie lassen sich *exakt* als Summe nach Gleichung (1) darstellen (Bestimmtheitsmaß $R^2 = 1,000$, Abschnitt 9).

Symbol	Bedeutung
N	Anzahl der Spuren eines Stücks (variabel, hier 4–37)
$s_i(t)$	Audiosignal der Spur i (mono)
$e_i \in \mathbb{R}^{128}$	Klang-Embedding der Spur i (VGGish)
$c \in \mathbb{R}^{128}$	Kontextvektor: mittleres Embedding aller Spuren
$g_i \geq 0$	Gain (linear), in dB: $20 \log_{10} g_i$
$\theta_i \in [0, \pi/2]$	Panoramawinkel: 0 = ganz links, $\pi/4$ = Mitte, $\pi/2$ = ganz rechts
$m_i \in \{0, 1\}$	Maske: 1 = echte Spur, 0 = Auffüll-Spur im Batch
$\hat{y}, y$	vorhergesagter Stereomix, Referenzmix

Tabelle 1: Notation.

2 Überblick: Der Weg durch das Modell

Abbildung 1 zeigt die gesamte Kette. Sie besteht aus vier Bausteinen, von denen *nur einer* trainiert wird:

1. **VGGish-Encoder** (eingefroren): beschreibt den Klang jeder Spur als Vektor e_i .
2. **Kontext** (keine Parameter): Mittelwert aller e_i – „was spielt in diesem Stück sonst noch mit?“
3. **MLP** (trainierbar): entscheidet aus e_i und Kontext c über Gain g_i und Pan θ_i .
4. **Mixer** (keine Parameter): wendet die Einstellungen nach Gleichung (1) an und summiert zum Stereomix.

Im Training vergleicht eine Verlustfunktion den Mix mit einer Referenz; der Fehler fließt als Gradient durch den Mixer zurück in das MLP.

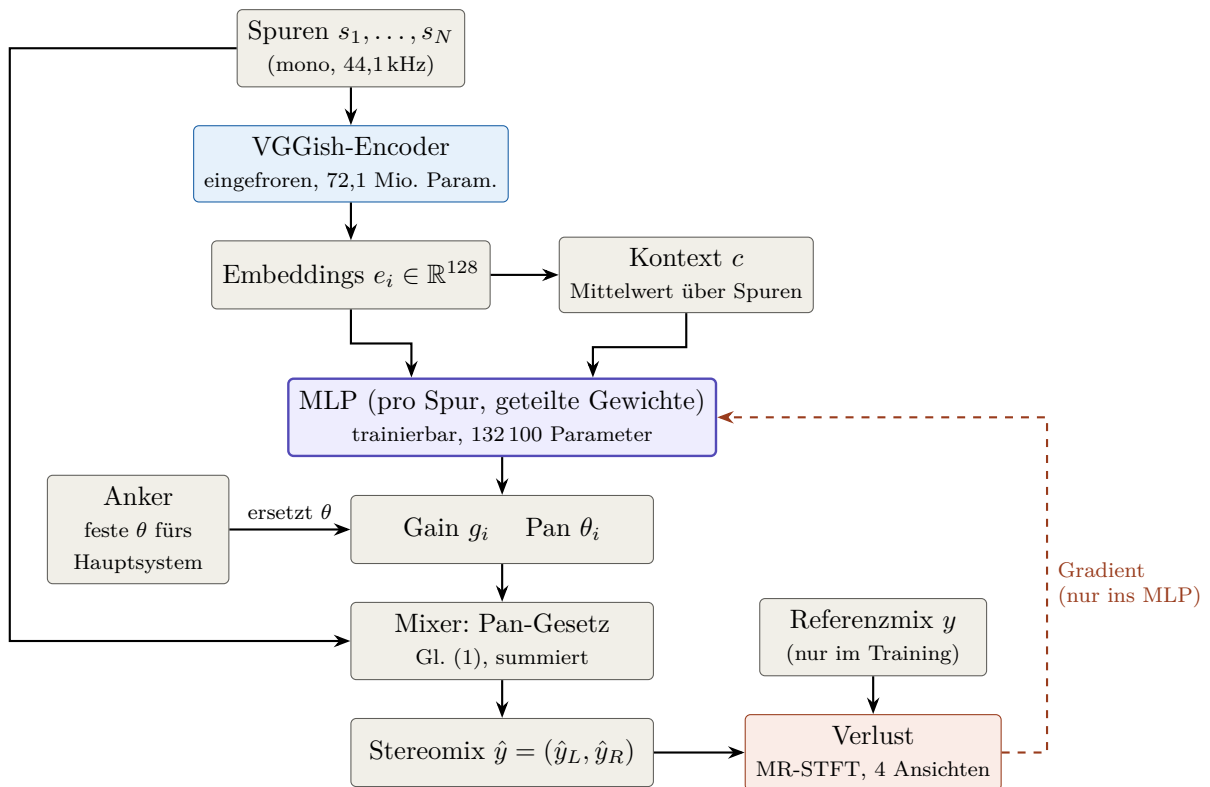


Abbildung 1: Die Pipeline. Blau: eingefroren. Violett: der einzige trainierte Baustein. Grau: feste Rechenschritte ohne Parameter. Die Rohspuren gehen zweimal ein: einmal zur Klangbeschreibung, einmal direkt in den Mixer.

Kernaussage. Das Modell erzeugt keinen Klang, es erzeugt Reglerstellungen. Das Audio selbst wird nur skaliert und im Panorama verteilt. Dadurch kann das Modell keine Artefakte „erfinden“, und seine Entscheidungen sind so lesbar wie ein Mischpult.

3 Schritt 1: Vom Audio zum Klangvektor – der VGGish-Encoder

Damit das Netz entscheiden kann, *was* auf einer Spur zu hören ist, muss das Audio in eine kompakte Beschreibung übersetzt werden. Dafür verwenden wir VGGish [2], ein Faltungsnetz, das auf einer sehr großen Menge von Videotonspuren darauf trainiert wurde, Geräusch- und Musikklassen zu erkennen.

Ablauf für eine Spur.

1. Die Spur wird von 44,1 kHz auf 16 kHz umgetastet (VGGish erwartet diese Rate).
2. Sie wird in Abschnitte von 0,96 s geschnitten; für jeden Abschnitt wird ein logarithmisches Mel-Spektrogramm berechnet, also eine Darstellung der Energie über Zeit und gehörangepassten Frequenzbändern.
3. Das Faltungsnetz bildet jeden Abschnitt auf einen Vektor mit 128 Zahlen ab.
4. Die Vektoren aller Abschnitte werden gemittelt. Ergebnis: *ein* Vektor $e_i \in \mathbb{R}^{128}$ pro Spur.

Warum eingefroren? VGGish hat rund 72,1 Millionen Parameter. Mit unseren wenigen Trainingsdaten (Abschnitt 9) ließen sich diese nicht sinnvoll nachtrainieren – das Netz würde die Trainingsstücke auswendig lernen. Eingefroren liefert VGGish dagegen eine robuste, allgemeine Klangbeschreibung, und trainiert wird nur der kleine Entscheidungsteil.

Was im Vektor steckt. Vor allem die *Klangfarbe*: Eine Geigen-Spur und eine Posaunen-Spur landen an deutlich verschiedenen Stellen. Zum Teil auch der *Pegel*: Wird dieselbe Spur um 24 dB leiser gemacht, verschiebt sich der Vektor spürbar (Abstand 0,93 bei einer Vektorlänge von etwa 2,5). Das ist wichtig, denn ohne Pegelinformation könnte das Modell keinen passenden Gain wählen.

Was verloren geht. Durch die Mittelung über die Zeit weiß das Modell nicht, *wann* ein Instrument spielt oder wie sich die Dynamik entwickelt – passend zur Entscheidung für statische Einstellungen, aber eine klare Grenze (Abschnitt 12).

4 Schritt 2: Der Kontext – was spielt sonst noch mit?

Wie laut und wo eine Geige gemischt wird, hängt davon ab, was sonst im Stück passiert: Im Streichquartett ist sie Hauptstimme, in der Sinfonie eine von vielen. Deshalb bekommt jede Spur zusätzlich einen *Kontextvektor*, den Mittelwert über alle Spuren des Stücks:

$$c = \frac{\sum_{i=1}^N m_i e_i}{\sum_{i=1}^N m_i}. \quad (2)$$

Wozu die Maske m_i ? Im Training werden mehrere Stücke gleichzeitig verarbeitet (ein *Batch*). Weil Stücke unterschiedlich viele Spuren haben, werden kürzere Listen mit stummen Platzhalterspuren aufgefüllt. Die Maske sorgt dafür, dass diese Platzhalter weder in den Mittelwert noch später in den Mix eingehen.

Zwei nützliche Eigenschaften.

- **Beliebige Spurzahl:** Ein Mittelwert funktioniert für 4 wie für 37 Spuren.
- **Reihenfolge egal:** Vertauscht man die Spuren, ändert sich c nicht, und jede Spur bekommt dieselbe Entscheidung. Das Modell ist *permutationsäquivalent* – die Dateireihenfolge spielt keine Rolle.

5 Schritt 3: Die Entscheidung – das MLP

Das mehrschichtige Perzeptron (MLP) ist der einzige Teil, der lernt. Es bekommt für jede Spur ihren eigenen Klangvektor und den Kontext, hintereinander gehängt zu einem Vektor mit 256

Zahlen, und gibt zwei Zahlen aus.

Schicht	Form	Parameter
Eingabe $[e_i; c]$	256	–
Linear + PReLU + Dropout 0,1	$256 \rightarrow 256$	65 793
Linear + PReLU + Dropout 0,1	$256 \rightarrow 256$	65 793
Linear	$256 \rightarrow 2$	514
Summe		132 100

Tabelle 2: Architektur des MLP. PReLU ist eine Aktivierungsfunktion mit lernbarer Steigung im negativen Bereich; Dropout schaltet im Training zufällig 10 % der Neuronen ab und beugt so dem Auswendiglernen vor.

Die beiden Rohausgaben z_1, z_2 werden in gültige Reglerstellungen umgerechnet:

$$g_i = \text{softplus}(z_1) = \ln(1 + e^{z_1}) \geq 0, \quad \theta_i = \frac{\pi}{2} \sigma(z_2) = \frac{\pi}{2} \cdot \frac{1}{1 + e^{-z_2}} \in [0, \frac{\pi}{2}]. \quad (3)$$

Softplus garantiert einen nicht-negativen Gain, die Sigmoid-Funktion σ einen Winkel zwischen ganz links und ganz rechts. Zu Beginn des Trainings liegen die Ausgaben nahe null, also $\theta \approx \pi/4$: Jede Spur startet in der Mitte.

Geteilte Gewichte. Für jede Spur wird *dasselbe* MLP verwendet. Das Modell lernt also keine Regel „Spur Nummer 3 nach links“, sondern „eine Spur, die *so* klingt, in einem Stück, das *so* klingt, gehört dorthin“. Nur deshalb funktioniert es auf Aufnahmen mit anderen Spurnamen und anderer Spurzahl.

6 Schritt 4: Anker – feste Positionen für das Hauptsystem

Bei den Mikrofonen des Hauptsystems ist die Stereoposition physikalisch vorgegeben: Das linke Tree-Mikrofon gehört nach links, das rechte nach rechts. Diese Spuren werden über ihren Dateinamen erkannt (z. B. *_L, *_C, *_R, *_RoomL) und bekommen einen festen Winkel: 0°, 45° bzw. 90°. Der vorhergesagte Winkel wird für sie verworfen; ihren *Gain* wählt das Modell aber weiterhin selbst.

Anker haben zwei Vorteile: Sie geben dem Stereobild eine eindeutige Orientierung (Abschnitt 8) und nehmen dem Modell Entscheidungen ab, die ohnehin feststehen. Wichtig in der Anwendung: Beim Mischen eines neuen Stücks muss dieselbe Ankerliste gelten wie im Training.

7 Schritt 5: Der Mixer – ein differenzierbares Pan-Gesetz

Der Mixer setzt Gleichung (1) um. Der Anteil einer Spur im linken und rechten Kanal ist $g \cos \theta$ bzw. $g \sin \theta$. Weil $\cos^2 \theta + \sin^2 \theta = 1$ gilt, bleibt die abgestrahlte *Leistung* einer Spur an jeder Panoramaposition gleich (*Constant-Power-Panning*): Ein Instrument wird beim Verschieben nicht lauter oder leiser. In der Mitte ($\theta = 45^\circ$) erhält jeder Kanal den Faktor $\cos 45^\circ \approx 0,707$, also -3 dB (Abbildung 2).

Kernaussage. Der Mixer ist die entscheidende Idee des Ansatzes. Weil er aus lauter differenzierbaren Operationen besteht (Multiplikationen, Kosinus, Sinus, Summen), lässt sich berechnen, wie sich der Fehler des fertigen Mixes ändert, wenn man einen Regler ein wenig verstellt. Das Modell braucht deshalb *keine* Beispiele für „richtige“ Reglerstellungen – es lernt allein aus Paaren von Einzelspuren und fertigem Mix. Diese Idee eines differenzierbaren Mischpults geht auf Steinmetz et al. [1] zurück; unser Mischpult ist bewusst auf Gain und Pan reduziert.

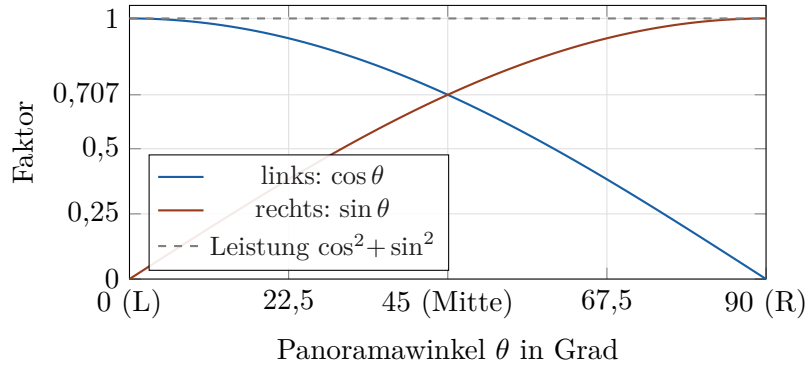


Abbildung 2: Das Constant-Power-Pan-Gesetz. Die Gesamtleistung einer Spur (grau gestrichelt) ist unabhängig von ihrer Position.

8 Schritt 6: Wie gut ist der Mix? – die Verlustfunktion

Die Verlustfunktion misst, wie weit der vorhergesagte Mix $\hat{y}$ von der Referenz y entfernt ist. Ein direkter Vergleich der Wellenformen Sample für Sample wäre ungeeignet: Er reagiert extrem empfindlich auf kleinste Phasenunterschiede, die man nicht hört. Stattdessen vergleichen wir *Spektrogramme*, also die Energie über Zeit und Frequenz.

Ein Spektrogramm-Vergleich. Für ein Signal wird mit der Kurzzeit-Fouriertransformation (STFT, Hann-Fenster der Länge n , Schrittweite $n/4$) das Betragsspektrogramm $|X|$ berechnet. Zwei Maße werden addiert:

$$\mathcal{L}_{\text{SC}} = \frac{\| |Y| - |\hat{Y}| \|_F}{\| |Y| \|_F}, \quad (4)$$

$$\mathcal{L}_{\text{LM}} = \frac{1}{K} \left\| \log(|Y| + \varepsilon) - \log(|\hat{Y}| + \varepsilon) \right\|_1. \quad (5)$$

Die *Spectral Convergence* $\mathcal{L}_{\text{SC}}$ misst die relative Abweichung und wird von den lauten Anteilen dominiert. Der *Log-Magnitude*-Term $\mathcal{L}_{\text{LM}}$ vergleicht logarithmierte Beträge; er gewichtet leise Anteile stärker und liegt näher an der Lautheitswahrnehmung des Gehörs. $\| \cdot \|_F$ ist die Frobenius-Norm, K die Anzahl der Spektrogramm-Einträge, $\varepsilon = 10^{-8}$.

Mehrere Auflösungen. Ein langes Fenster löst Frequenzen fein auf, aber Zeit grob – ein kurzes umgekehrt. Deshalb wird der Vergleich bei drei Fenstergrößen $n \in \{512, 1024, 2048\}$ gerechnet und gemittelt (*Multi-Resolution-STFT-Verlust*, MR-STFT [3, 4]):

$$\mathcal{L}_{\text{MR}}(\hat{x}, x) = \frac{1}{3} \sum_{n \in \{512, 1024, 2048\}} \left(\mathcal{L}_{\text{SC}}^{(n)} + \mathcal{L}_{\text{LM}}^{(n)} \right). \quad (6)$$

Vier Ansichten des Stereosignals. Der Gesamtverlust vergleicht vier aus dem Stereosignal abgeleitete Signale:

$$\begin{aligned} \mathcal{L} = & \underbrace{\mathcal{L}_{\text{MR}}(\hat{y}_L + \hat{y}_R, y_L + y_R)}_{\text{Summe (Mitte)}} + \underbrace{\mathcal{L}_{\text{MR}}(\hat{y}_L - \hat{y}_R, y_L - y_R)}_{\text{Differenz (Breite)}} \\ & + \underbrace{\mathcal{L}_{\text{MR}}(\hat{y}_L, y_L)}_{\text{links}} + \underbrace{\mathcal{L}_{\text{MR}}(\hat{y}_R, y_R)}_{\text{rechts}}. \end{aligned} \quad (7)$$

Summe und Differenz stammen aus [1]. Die beiden Kanalterme haben wir ergänzt, und sie sind für Orchester entscheidend:

Das Spiegel-Problem. Vertauscht man in einem Mix links und rechts, bleibt die Summe $L + R$ gleich, und die Differenz $L - R$ wechselt nur das Vorzeichen – ihr Betragsspektrum ist identisch. Ein Verlust, der nur Summe und Differenz vergleicht, kann einen Mix also nicht von seinem Spiegelbild unterscheiden. Für Pop-Musik, wo die Seitenwahl oft beliebig ist, ist das gewollt [1]. Im Orchester gibt es dagegen eine feste Sitzordnung. In unseren frühen Versuchen hat das Modell diese Blindheit ausgenutzt und fast alles auf eine Seite gelegt (Pegelunterschied links zu rechts bis 22,7 dB). Die Kanalterme in Gleichung (7) bestrafen Energie auf der falschen Seite und beheben dieses Kollabieren; die Anker (Schritt 4) wirken zusätzlich strukturell in dieselbe Richtung.

9 Schritt 7: Das Training

9.1 Daten

Datensatz	Beschreibung	Umfang
Spheres	Echte Aufnahmen mit echtem Tonmeister-Mix (Hauptsystem L/C/R, Raum-Paar, Spots). Die Mischungen sind exakt statische Gain/Pan-Summen ($R^2 = 1,000$); je zwei Songs teilen dieselben Einstellungen.	4 Songs, 2 Mischungen, 5,7 min
SynthSOD	Synthetisch gerenderte Orchesterstücke, jedes Instrument einzeln pro Mikrofonposition (Close Mic ohne Übersprechen, Tree als Hauptsystem).	576 Stücke, davon 64 genutzt
Synthmix	Unsere Neuberechnung der SynthSOD-Ziele: Spots nach einem festen Sitzplan im Panorama verteilt (plus $\pm 5^\circ$ Zufall), Gains zufällig. Die wahren Werte werden pro Stück gespeichert.	64 Stücke

Tabelle 3: Trainingsdaten. Die Synthmix-Ziele machen die Genauigkeit der Positionen direkt messbar, weil die richtige Antwort bekannt ist.

Der Sitzplan von Synthmix folgt der üblichen Orchesteraufstellung aus Zuhörersicht (Violine 1 15° , Violine 2 30° , Viola 55° , Cello 70° , Kontrabass 80° , Holzbläser um die Mitte, Blech halbrechts). Die zurückgerechneten Einstellungen der beiden echten Spheres-Tonmeister bestätigen ihn weitgehend: Violine 1 bei $9,5^\circ$ bis 16° , Viola bei 57° bis 59° , Horn bei 30° bis 31° .

9.2 Ablauf

- **Aufteilung nach Stücken:** 10 % der Stücke (7 von 68) werden für die Validierung zurückgehalten – ganze Stücke, nicht einzelne Ausschnitte, damit das Modell dort wirklich Unbekanntes sieht.
- **Ausschnitte:** Pro Epoche werden 1 000 zufällige Ausschnitte von 5 s Länge gezogen; zur Validierung dienen feste 200 Ausschnitte.
- **Optimierung:** Adam [5] mit Lernrate $3 \cdot 10^{-4}$, Batchgröße 16. Verbessert sich der Validierungsverlust 20 Epochen lang nicht, wird die Lernrate halbiert.
- **Nur das MLP** wird optimiert; VGGish bleibt unverändert.
- **Rechenzeit:** Das Training lief auf einer Kaggle-GPU, etwa 2,5 min pro Epoche.

9.3 Nächster Schritt: Die echten Mischungen stärker nutzen

Bei gleichverteilter Auswahl stammen nur etwa 7 % der Ausschnitte aus Spheres. Im laufenden Nachfolge-Experiment bekommt Spheres 40 % der Ausschnitte, und zwei Datenaugmentierungen

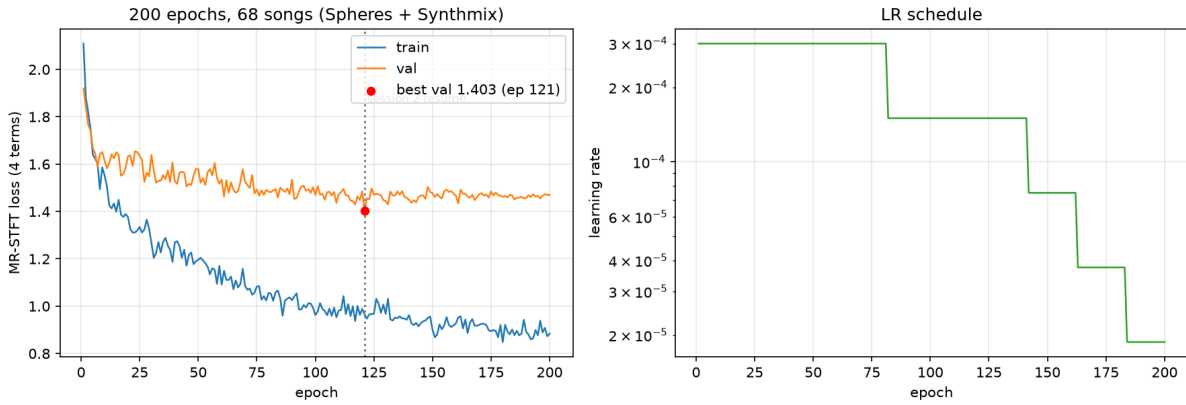


Abbildung 3: Verlustkurven des 200-Epochen-Laufs. Der beste Validierungsverlust wird in Epoche 121 erreicht; danach sinkt nur noch der Trainingsverlust (leichtes Overfitting).

erweitern die zwei echten Mischungen:

- **Pegelvariation:** Jede Spur wird beim Laden zufällig um bis zu ± 6 dB lauter oder leiser gemacht, das Ziel bleibt gleich. Der richtige Gain verschiebt sich damit um den Gegenwert – das Modell muss den Pegel aus dem Audio ablesen.
- **Spur-Dropout:** Zufällig weggelassene Spots werden mit ihrem exakten Anteil vom Ziel abgezogen. Aus zwei Mischungen entstehen so viele gültige Teilbesetzungen.

10 Schritt 8: Anwendung auf ein neues Stück

Nach dem Training wird ein neues Stück so gemischt:

1. Alle Spuren werden geladen; Spuren des Hauptsystems erhalten ihre Anker-Winkel.
2. VGGish beschreibt jede Spur über ihre *gesamte* Länge (Mittelwert über alle 0,96 s-Abschnitte des Stücks).
3. Das MLP liefert einmal Gain und Pan pro Spur.
4. Der Mixer wendet diese festen Einstellungen auf das ganze Stück an.

Das Ergebnis ist ein vollständiger Stereomix und zugleich eine lesbare Mischpult-Einstellung (Abbildung 4).

11 Was das Modell kann – Ergebnisse im Überblick

Positionen auf unbekannten Stücken (Synthmix-Validierung). Gemessen auf den 7 zurückgehaltenen Stücken mit bekannter Aufstellung, 32 Spots:

Maß	60-Epochen-Lauf	200-Epochen-Lauf	Alles in die Mitte	Bestmöglich
Pan-Fehler (Mittel)	22,4°	5,8°	17,1°	2,5°
Stereobalance $ L - R $	1,79 dB	1,29 dB	–	1,39 dB (Ziel)

Tabelle 4: Die Untergrenze von 2,5° ergibt sich aus dem $\pm 5^\circ$ -Zufall im Sitzplan, der aus dem Audio grundsätzlich nicht vorhersagbar ist.

Das Modell reproduziert die Sitzordnung monoton (Violine 1 links bis Kontrabass rechts), mit einer leichten Stauchung zur Mitte: Äußere Instrumente landen etwas zu weit innen.

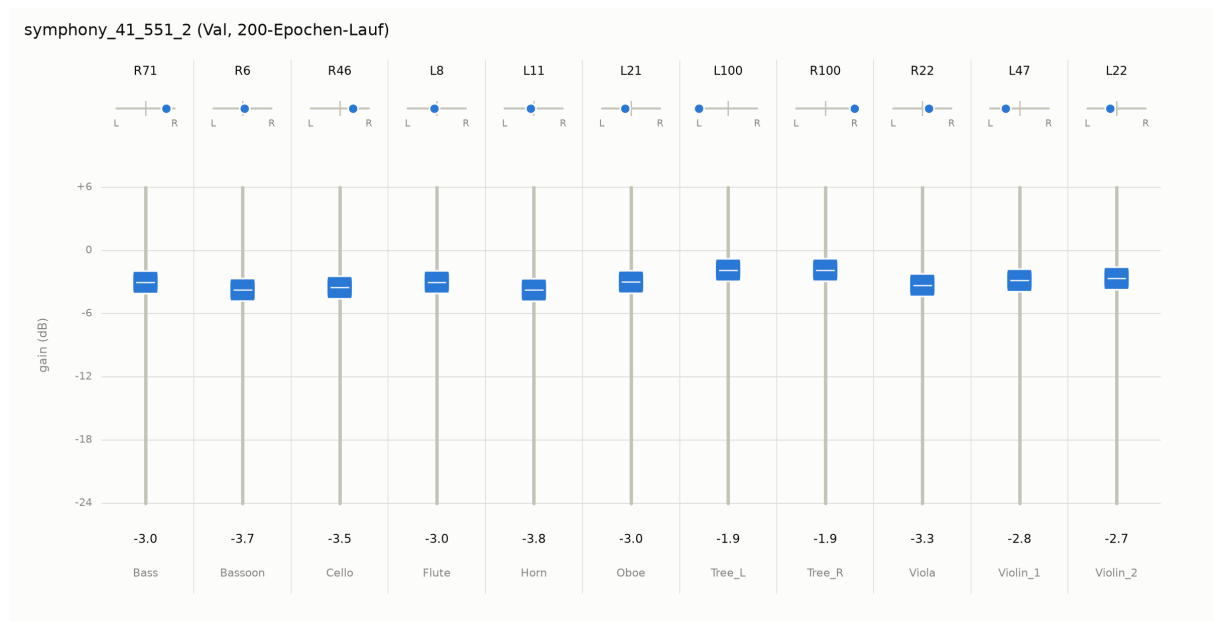


Abbildung 4: Vom Modell gewählte Einstellungen für ein zurückgehaltenes Stück (Sinfonie, 11 Spuren): oben die Panoramaposition, darunter der Fader. Die Spuren Tree_L und Tree_R (Hauptsystem) sind Anker.

Einstellungen echter Tonmeister (Spheres). Gegen die zurückgerechneten Werte der zwei echten Mischungen erreicht das Modell einen Pan-Fehler von $8,2^\circ$ (Mitte-Tipp: $16,3^\circ$) und einen Gain-Fehler von $2,97\text{ dB}$ (konstanter Pegel: $5,15\text{ dB}$). Achtung: Spheres war im Training, dies ist also keine Messung auf Unbekanntem.

Echte Konzertaufnahme (Wuppertal, 33–37 Spuren). Qualitativ bewertet: In einem lyrischen Ausschnitt ordnet das Modell die Streicher korrekt von links nach rechts; in dichten Passagen staucht es fast alle Spots auf 45° bis 60° zusammen und wählt nahezu einheitliche Pegel.

12 Grenzen und offene Punkte

- **Gains aus Synthmix sind nicht lernbar.** Die Spot-Pegel werden dort zufällig gewürfelt, unabhängig vom Klang. Das Modell kann daraus nur den Mittelwert lernen (Gain-Fehler $1,37\text{ dB}$ gegenüber $1,5\text{ dB}$ für einen konstanten Tipp). Echte Pegelentscheidungen enthält nur Spheres, und dort unterscheiden sie sich stark zwischen den Stücken: Bei Mozart stehen die Streicher vorn, bei Tschaikowsky die Holzbläser.
- **Stauchung zur Mitte.** Bei Unsicherheit ist die Mitte der sicherste Tipp; das Modell nutzt die Stereobreite nicht voll aus.
- **Abstand zu echten Aufnahmen.** Die synthetischen Spots haben kein Übersprechen, echte schon; im Training gab es 4–22 Spuren, in Wuppertal 33–37, teils mit zwei Mikrofonen pro Stimmgruppe.
- **Statische Einstellungen.** Keine Pegelautomation, kein EQ, kein Hall, keine Laufzeitkorrektur zwischen Spot und Hauptsystem.
- **Wenig echte Referenzen.** Zwei echte Mischungen reichen nicht, um stückabhängige

Pegelentscheidungen zu lernen, und es gibt bisher keine zurückgehaltene echte Mischung als Testfall.

13 Vorschlag: Vom Dokument zum Paper

Paper-Abschnitt	Inhalt aus diesem Dokument
Einleitung	Abschnitt 1: Aufgabe, Hauptsystem vs. Spots, Motivation für statische Gain/Pan-Entscheidungen.
Verwandte Arbeiten	Differenzierbares Mischpult [1], VGGish [2], MR-STFT-Verlust [3, 4]; L/R-Invarianz bei Steinmetz vs. feste Orchesteraufstellung.
Methode	Abschnitte 3–7 und Abbildung 1; Gleichungen (1), (2), (7).
Daten	Abschnitt 9: Spheres, SynthSOD, Synthmix; Sitzplan und dessen Bestätigung durch echte Tonmeister.
Experimente	Pan-Kollaps und dessen Behebung (Verlust mit Kanaltermen), 200-Epochen-Lauf, Spheres-Gewichtung.
Ergebnisse	Pan-Fehler mit Baselines und Rauschgrenze; Spheres; Wuppertal als qualitative Fallstudie.
Diskussion	Abschnitt 12.

Noch zu ergänzen. Quellenangaben für den Spheres- und den SynthSOD-Datensatz (in diesem Dokument noch nicht zitiert), die Ergebnisse des laufenden Spheres-Experiments sowie Hörbeispiele.

Literatur

- [1] C. J. Steinmetz, J. Pons, S. Pascual, J. Serrà: *Automatic multitrack mixing with a differentiable mixing console of neural audio effects*. Proc. IEEE ICASSP, 2021.
- [2] S. Hershey et al.: *CNN architectures for large-scale audio classification*. Proc. IEEE ICASSP, 2017.
- [3] R. Yamamoto, E. Song, J.-M. Kim: *Parallel WaveGAN: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram*. Proc. IEEE ICASSP, 2020.
- [4] S. Ö. Arik, H. Jun, G. Diamos: *Fast spectrogram inversion using multi-head convolutional neural networks*. IEEE Signal Processing Letters 26(1), 2019.
- [5] D. P. Kingma, J. Ba: *Adam: A method for stochastic optimization*. Proc. ICLR, 2015.